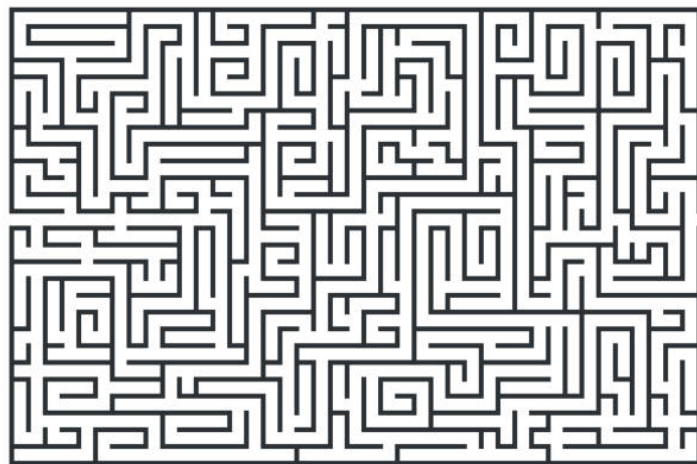


Correction devoir Maison 2025-2026

Labyrinthes, Machines de Turing et

Complexité



Nicolas VACELET – Université Marie et Louis Pasteur

13 Janvier 2026

1 Modéliser un labyrinthe avec un graphe

1. Exemple simple de labyrinthe

On considère un labyrinthe de 9 cases, organisé sous forme d'une grille 3×3 :

Le labyrinthe peut être représenté graphiquement ainsi :

S	.	#
#	.	#
#	.	F

- S : case de départ
- F : case d'arrivée
- $.$: case libre
- $\#$: mur

Les cases libres sont :

$$C = \{(1, 1), (1, 2), (2, 2), (3, 2), (3, 3)\}$$

2. Transformation du labyrinthe en graphe

On transforme maintenant le labyrinthe en un **graphe non orienté** :

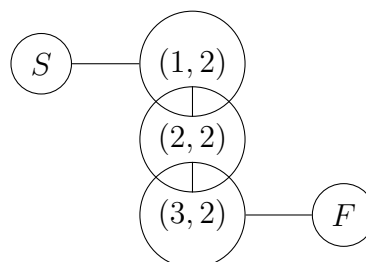
- chaque case libre devient un **sommet** ;
- deux sommets sont reliés par une **arête** si les cases correspondantes sont voisines (haut, bas, gauche, droite).

Deux cases (i, j) et (i', j') sont voisines si :

$$|i - i'| + |j - j'| = 1$$

Cette formule correspond à la **distance de Manhattan**.

Le graphe correspondant au labyrinthe est alors :



Chaque sommet représente une case libre, et chaque arête un déplacement possible.

3. Matrice d'adjacence

En numérotant les sommets dans l'ordre (c'est à dire la representation des cases libres)

$$S, (1, 2), (2, 2), (3, 2), F$$

la matrice d'adjacence du graphe représente par un 1 les déplacements possibles :

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Ainsi, une fois le labyrinthe transformé en graphe $G = (C, E)$, résoudre le labyrinthe revient à chercher un chemin dans le graphe reliant S à F, en traversant les cases de C, via les arêtes de E.

Le problème concret du labyrinthe devient ainsi un problème mathématique de recherche de chemin dans un graphe.

2 Machine de Turing et décidabilité

On cherche maintenant à aller plus loin : peut-on résoudre un labyrinthe à l'aide d'une machine de Turing et savoir si ce problème est décidable ?

1. Encodage du labyrinthe sur le ruban

Comme vu dans le cours, une machine de Turing ne manipule que des symboles sur un ruban infini. Il faut donc commencer par trouver une façon d'encoder le labyrinthe sous forme d'une suite de symboles sur un ruban. Pour cela, on définit les éléments qui constituent une MT, l'alphabet.

Alphabet d'entrée :

$$\Sigma = \{0, 1, S, F, \#\}$$

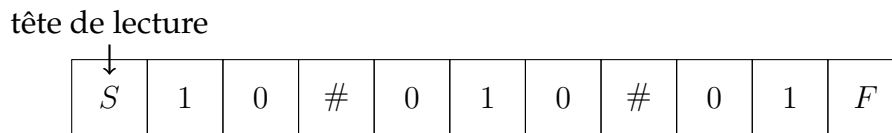
- 0 : mur
- 1 : case libre
- S : case de départ
- F : case d'arrivée
- # : séparateur entre les lignes du labyrinthe

Il faut distinguer l'alphabet d'entrée, de l'alphabet que la machine utilisera pour fonctionner

$$\Gamma = \Sigma \cup \{V, B\}$$

- V : case déjà visitée
- B : case blanche (zone non utilisée du ruban)

Notre labyrinthe peut être encodé de cette façon :



La machine de Turing ne « comprend » pas le labyrinthe, elle applique mécaniquement des règles de transition entre des **états**.

États principaux

- q_0 : recherche de la position de départ S sur le ruban
- q_1 : parcours du labyrinthe encodé
- q_{test} : test des cases voisines (haut, bas, gauche, droite)
- q_{reset} : retour au début du ruban
- q_f : état final acceptant (succès)

Principe de fonctionnement

- La machine commence en q_0 et se déplace sur le ruban jusqu'à trouver le symbole S .
- Elle marque cette case comme visitée en la remplaçant par V .
- En état q_{test} , elle cherche les cases voisines :
 - si une case voisine contient 1, elle s'y déplace ;
 - si une case voisine contient F , la machine passe en q_f et s'arrête.
- Chaque case visitée est marquée par V pour éviter de revenir en arrière indéfiniment.
- Si aucune nouvelle case n'est accessible, la machine revient au début du ruban (q_{reset}) et recommence l'exploration.

Donc, la MT permet une exploration systématique du labyrinthe, comparable à un algorithme de parcours de graphe.

Comment une machine de Turing teste le voisinage d'une case

Bien que le labyrinthe (en deux dimensions) soit encodé de façon linéaire (en une dimension) sur le ruban, la machine de Turing est capable de retrouver les *voisins* d'une case par des déplacements successifs de sa tête de lecture.

- La machine connaît la longueur d'une ligne du labyrinthe.
- À partir d'une case courante, elle peut :
 - se déplacer d'une case à gauche ou à droite sur le ruban pour tester les voisins horizontaux ;
 - se déplacer de plusieurs cases vers la gauche ou la droite pour franchir un séparateur #, puis se déplacer du même nombre de cases pour atteindre la case située au-dessus ou en dessous.

Ainsi, la machine de Turing peut tester le voisinage d'une case du labyrinthe uniquement à l'aide de déplacements simples sur son ruban.

2. Décidabilité du problème

« Existe-t-il un algorithme qui s'arrête toujours et qui décide si un labyrinthe admet un chemin entre S et F ? »

Cas d'un labyrinthe fini

- Le nombre de cases est fini.
- Le nombre de configurations possibles de la machine de Turing est donc fini.
- En marquant les cases visitées, la machine ne peut pas explorer indéfiniment.

⇒ Le problème est **décidable**.

La machine s'arrête toujours, soit en acceptant (chemin trouvé), soit en refusant.

Cas d'un labyrinthe infini

- Le labyrinthe contient une infinité de cases.
- La machine peut continuer à explorer sans jamais atteindre F .
- Il est impossible de garantir qu'elle s'arrête si aucun chemin n'existe.

⇒ Le problème est **semi-décidable**.

Cela signifie que :

- si un chemin existe, la machine de Turing finira peut-être par l'atteindre et s'arrêtera ;
- si aucun chemin n'existe, la machine peut tourner indéfiniment.

3 Algorithmes de résolution de labyrinthes

Une fois le labyrinthe modélisé sous forme de graphe, différents algorithmes peuvent être utilisés pour trouver (ou non) un chemin entre l'entrée S et la sortie F . Ces algorithmes n'ont pas tous le même comportement, ni les mêmes garanties.

Méthode de Pledge

- on choisit une direction globale (par exemple « vers le nord ») ;
- lorsqu'un obstacle est rencontré, on suit un mur ;
- on compte les rotations effectuées (droite : $+90^\circ$, gauche : -90°) ;
- dès que la somme des rotations redevient nulle, on quitte le mur.

Le compteur d'angles permet de ne pas rester bloqué dans des boucles locales. Le chemin trouvé n'est pas forcément optimal ; La méthode dépend fortement de la géométrie du labyrinthe.

Algorithme de Trémaux

- chaque passage (arête) est marqué lors de son utilisation ;
- un passage marqué deux fois n'est plus emprunté ;
- lorsqu'on arrive à un cul-de-sac, on fait demi-tour.
- aucun cycle n'est parcouru indéfiniment ;
- si une sortie existe, elle sera forcément trouvée.

Algorithme de Tarry

Principe :

- exploration systématique de toutes les arêtes ;
- chaque arête est parcourue exactement une ou deux fois ;
- garantit une exploration complète du labyrinthe.
- peut être très long ;
- ne cherche pas un chemin court.

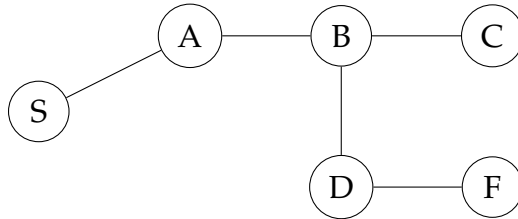
Algorithmes de parcours de graphes

Lorsque le labyrinthe est modélisé par un graphe $G = (V, E)$, on peut utiliser des algorithmes standards.

Parcours en profondeur (DFS)

Principe :

- on part de S ;
- on choisit un voisin non visité ;
- on continue tant que possible ;
- lorsqu'on est bloqué, on revient en arrière (backtracking).



Pour cet exemple de graphe, un parcours DFS possible :

$$S \rightarrow A \rightarrow B \rightarrow C$$

Blocage en C :

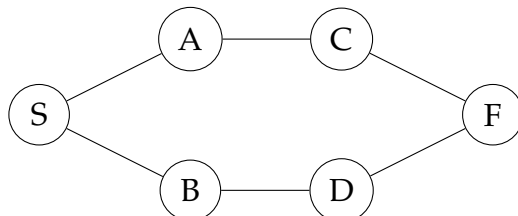
$$\text{retour } B \rightarrow D \rightarrow F$$

Parcours en largeur (BFS)

Principe :

- on explore tous les sommets à distance 1 de S ;
- puis ceux à distance 2 ;
- etc.

la première fois que F est atteint, le chemin est le plus court.



Pour cet exemple de graphe, un parcours BFS possible :

$$\{A, B\} \rightarrow \{C, D\} \rightarrow \{F\}$$

4 Complexité algorithmique

```
BFS(G, S, F):  
  f <- file vide  
  couleur[u] <- BLANC pour tout u  
  parent[u] <- NULL pour tout u  
  f.ajouter(S)  
  couleur[S] <- GRIS  
  tant que f non vide:  
    u <- f.retirer()  
    si u == F: retourner vrai  
    pour chaque voisin v de u:  
      si couleur[v] == BLANC:  
        couleur[v] <- GRIS  
        parent[v] <- u  
        f.ajouter(v)  
    couleur[u] <- NOIR  
  retourner faux
```

Le BFS (parcours en largeur) explore systématiquement toutes les cases accessibles du labyrinthe, modélisé par un graphe $G=(V,E)$, où V est l'ensemble des cases libres et E l'ensemble des passages entre elles.

Temps :

Chaque case libre (V) est visitée exactement une fois. À chaque visite, on marque la case et on la retire de la file. Ces opérations sont donc proportionnelles à V

Pour chaque case, on parcourt tous ses voisins. Comme chaque passage entre deux cases est représenté par une arête, et qu'une arête est examinée au plus deux fois (une fois depuis chaque extrémité), le coût lié aux voisins est proportionnel à E .

En combinant ces deux points, le nombre total d'opérations élémentaires est proportionnel à $V+E$ d'où une complexité temporelle de $O(V+E)$

Mémoire :

Le BFS utilise une file pour stocker les cases découvertes mais non encore explorées. Dans le pire des cas, cette file peut contenir toutes les cases du labyrinthe.

Il faut également mémoriser, pour chaque case, son état (non visitée, découverte, visitée) et éventuellement son parent pour reconstruire le chemin solution.

Ainsi, la mémoire nécessaire est proportionnelle au nombre de sommets V , soit $O(V)$.

5 Labyrinthes à contraintes

Certains labyrinthes ne se résolvent pas simplement en trouvant un chemin libre de S à F . On peut imposer des contraintes supplémentaires comme des clés à collecter, portes à franchir dans un certain ordre, limites de temps, etc. On peut en quelque sorte généraliser un labyrinthe avec un ensemble de contraintes.

- On a un graphe $G = (C, E)$ représentant les cases libres et les passages.
- S est le départ, F la sortie.

- K est l'ensemble des clés, P l'ensemble des portes à franchir, éventuellement avec d'autres contraintes (ordre, temps, etc.).

Question : Existe-t-il un chemin de S à F qui respecte toutes les contraintes ?

Un chemin candidat W de S à F peut être vérifié rapidement :

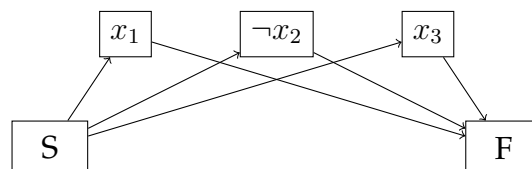
- chaque case du chemin est visitée,
- toutes les portes sont franchies seulement après avoir obtenu les clés correspondantes,
- toutes les contraintes temporelles ou d'ordre sont respectées.

Cette vérification nécessite un temps polynomial par rapport au nombre de cases et de passages. Donc le problème appartient à NP.

Réduction depuis SAT (problème de satisfiabilité)

On peut montrer la NP-complétude en réduisant un problème SAT ou 3-SAT à un labyrinthe à contraintes :

- On fait intervenir des variables booléennes x_i qui correspondent à des choix (aller à gauche, à droite, prendre une clé, ...).
- Pour chaque clause de la formule, on construit un **gadget** dans le labyrinthe : un sous-labyrinthe spécifique dont le passage nécessite de collecter au moins une clé correspondant à un littéral vrai de la clause. Chaque gadget encode ainsi la contrainte de satisfaction d'une clause.
- Résoudre le labyrinthe revient à trouver une combinaison de choix qui permet d'atteindre la sortie, ce qui correspond exactement à l'évaluation de la formule booléenne.
- Ainsi, trouver un chemin valide dans le labyrinthe correspond à trouver une assignation des variables qui satisfait toutes les clauses.
- La construction des gadgets et leur connexion se fait en temps polynomial par rapport à la taille de la formule, garantissant que la réduction 3-SAT $\rightarrow$ labyrinthe à contraintes est valide.



Gadget de clause : passage possible si au moins un littéral est vrai

Comme SAT est NP-complet, le labyrinthe à contraintes l'est aussi.

Autre possibilité, réduction depuis le problème Hamiltonien

Une autre réduction classique se fait depuis le problème du cycle Hamiltonien :

- Dans le problème Hamiltonien, on cherche un cycle passant exactement une fois par chaque sommet d'un graphe.
- On peut utiliser la représentation graphique du labyrinthe
- On impose des contraintes (par exemple : chaque salle doit être visitée exactement une fois avant d'atteindre la sortie, ou certaines portes obligent un ordre de passage).
- Trouver un chemin valide de S à F dans ce labyrinthe revient alors à résoudre le problème Hamiltonien.

Comme le problème Hamiltonien est NP-complet, résoudre un labyrinthe à contraintes l'est aussi.

Implications pour P vs NP

Classe P : ensemble des problèmes qui peuvent être résolus en temps polynomial. Autrement dit, il existe un algorithme efficace qui donne la solution pour toutes les instances.

Classe NP : ensemble des problèmes pour lesquels une solution candidate peut être vérifiée en temps polynomial, même si trouver cette solution peut être très long.

Notre problème de labyrinthe à contraintes appartient à NP :

Si on nous donne un chemin candidat, on peut vérifier en temps polynomial qu'il respecte toutes les contraintes (cases légales, portes et clés, ordre ou temps).

Cependant, trouver ce chemin parmi toutes les combinaisons possibles peut être très difficile, surtout si le labyrinthe est grand ou très contraint.

Est-ce que tous les problèmes dont on peut vérifier une solution rapidement peuvent aussi être résolus rapidement ?

Si $P=NP$, tout problème dont on peut vérifier la solution peut aussi être trouvé par un algorithme efficace.

Si $P \neq NP$, certains problèmes sont "faciles à vérifier" mais très difficiles à résoudre, et aucun algorithme rapide ne les résout en général.

Si $P=NP$, il existerait un algorithme polynomial capable de résoudre tous les labyrinthes à contraintes, même très grands et complexes, et par extension, les problèmes NP-complets.